CSE 516 · PROBABILISTIC GRAPHICAL MODELS · FINAL SUBMISSION (M4)

SPOT-The-Difference

Evaluation of Skeleton-Posterior-Guided Latent-Aware Causal Discovery on PetShop

Anant Narayan (AU2420009)
Ankit Kumar (AU2420147)
Milan Shah (AU2420195)

Group 6 (g6_th_apl_cybersecurity_team) · Ahmedabad University

Outline

1. Problem and Model
2. Method
3. Setup and Hypotheses
4. Results
5. Conclusions

Outline

1. Problem and Model
2. Method
3. Setup and Hypotheses
4. Results
5. Conclusions

Why this problem matters, and why a graphical model fits

Problem, importance, and the role of a PGM

The problem. Online applications are built from many small services (*microservices*) that call each other. When the application slows down, operators must find *which* service is the real cause.

Why it matters. Slow or failing services cost users and money, and a wrong diagnosis wastes hours. A map of which service influences which turns a flood of alerts into a short list of suspects.

Why a probabilistic graphical model.

- The *graph* states which services influence which: this is the structure we must learn
- The *probabilities* tell how strongly, and let us predict one service from others
- Hidden causes can be represented explicitly, which a plain DAG cannot do

Our task

Learn this graph from latency data; test whether a learned prior (SPOT) makes learning faster or more accurate than ABIC.

Terms used in this talk

Defined once here, used throughout

Term	Meaning
Latency	How long a service takes to answer a request (here: averaged response time over a time window)
Latent variable	A quantity that influences the data but is not measured, e.g. a service with no metric stream
ADMG	A graph with directed edges ($\rightarrow$, one service influences another) and bidirected edges ($\leftrightarrow$, a shared hidden cause), no cycles
Skeleton	The graph with arrow directions removed: only *whether* two services are connected
PAG	The summary graph that all equally-good ADMGs share; the target we score against
ABIC	Baseline learner that fits an ADMG by optimization, searching all d^2 pairs of d services
SPOT	ABIC guided by a learned estimate of which pairs are connected (the *skeleton posterior*, explained later)

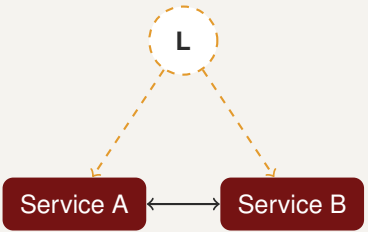
Hidden services make a plain DAG the wrong model

Why microservice latency needs an ADMG

Latency propagates. A slow database makes every service above it look slow; we want the origin, not the symptoms.

Some causes are never measured. Services without a usable metric stream become latent variables.

Use an ADMG, not a DAG. $\rightarrow$ for measured propagation, $\leftrightarrow$ for a shared unobserved cause.



A $\leftrightarrow$ B is how the model sees the hidden cause

Our question

Where does posterior guidance help, where does it not, and what limits it?

Linear-Gaussian ADMG, scored against the true PAG

Model and evaluation pipeline

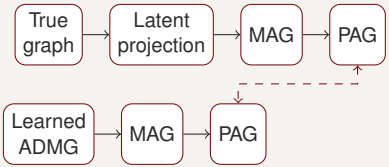
Linear-Gaussian ADMG

$$X = XW + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \Omega)$$

$$\Sigma = (I - W)^{-T}\,\Omega\,(I - W)^{-1}$$

- W: directed edges, callee $\rightarrow$ caller (reverse of call direction)
- Ω off-diagonal: bidirected edges from latent common causes
- Ancestral: no $\leftrightarrow$ between a node and its ancestor

How a learned graph is scored



Compared on: skeleton F1, arrowhead F1, tail F1, PAG SHD

PetShop: the real benchmark we test on

A microservice shop with a known call graph and injected slowdowns

- PetShop (Hardt et al., CLeaR 2024) is a public dataset from an online shop built from microservices
- It records latency for each service and provides the true call graph between services
- Four scenarios: low traffic, high traffic, temporal traffic 1 and 2
- Each scenario has 29–35 observed services and 9–13 hidden ones, so the true graph has latent confounding
- Separate runs inject a known slowdown into one service, used to test root-cause ranking

How we get the ground truth

Reverse the call graph (the callee causes the caller's slowdown), project out the hidden services, and convert to a PAG.

Limits to remember

One time series per scenario, so seeds measure optimizer variance only; data are not Gaussian i.i.d., so we rank-transform them first.

Outline

1. Problem and Model

2. Method

3. Setup and Hypotheses

4. Results

5. Conclusions

Differentiable causal discovery: from NOTEARS to ABIC

Learning a graph by gradient-based optimization instead of searching over graphs

Idea (NOTEARS, Zheng et al. 2018)

- The number of acyclic graphs explodes with d, so searching over graphs is hopeless
- Instead learn a weight matrix W ($W_{ij} \neq 0$ means $i \to j$) by fitting $X \approx XW$ with a sparsity penalty
- "No cycles" becomes one smooth equation:

$$h(W) = \operatorname{tr}\left(e^{W \circ W}\right) - d = 0$$

- $(W \circ W)^k_{ii}$ sums closed walks of length k back to node i; $h(W) = 0$ exactly when there is no directed cycle
- h is differentiable: minimize fit + penalty subject to $h(W) = 0$ with an augmented Lagrangian and gradient steps

ABIC (Bhattacharya et al. 2021)

- NOTEARS assumes every cause is measured; hidden causes create false edges
- ABIC learns an ADMG: weights W *and* an error covariance Ω whose off-diagonal entries are the bidirected edges ($\leftrightarrow$)
- Gaussian likelihood of (W, Ω) with a smooth sparsity penalty, keeping $h(W) = 0$ on the directed part
- A second smooth constraint, zero exactly when no $\leftrightarrow$ joins a node to one of its ancestors ("ancestral" graph)

The cost SPOT targets

Both learn d^2 weights, so every step touches all pairs. SPOT's posterior removes most pairs.

What is a “skeleton posterior”?

An estimate of how likely each pair of services is to be directly connected

- For every pair of services (i, j) it outputs $p_{ij} = P(i \text{ and } j \text{ are adjacent} \mid \text{data})$
- It is a classifier trained *offline* on simulated graphs where the true edges are known
- At run time it looks only at the data, never at the true graph

Two ways SPOT uses it

- **Mask:** updates to a pair are skipped at random according to p_{ij}
- **Hard floor:** pairs with p_{ij} below a threshold are fixed to zero

Calibrated means honest

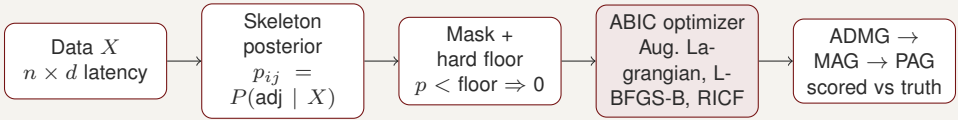
A posterior is *calibrated* if pairs given $p = 0.3$ are connected about 30% of the time (checked later with ECE and log-loss).

Why it speeds things up

Fewer candidate pairs means fewer parameters to fit.

SPOT is ABIC plus a learned skeleton posterior

The optimizer is shared on purpose



ABIC alone: no posterior, all d^2 candidate pairs are searched

Shared on purpose

Optimizer, objective, restarts, thresholds and seeds are identical for ABIC and SPOT.

Stochastic mask

A pair's gradient is masked by its posterior and the direction of the update.

Hard floor

Pairs with p below the floor are pinned to 0, which shrinks the search set.

The posterior is trained offline; the floor is calibrated

Three-stage gradient-boosting cascade, calibrated on simulated graphs

- **0** Markov-blanket features
- **1** Order-1 CI summaries for pairs passing 0.05
- **2** Order-2 CI summaries for pairs passing 0.10

Trained once, offline, on simulated ancestral ADMGs (d 25–40, n 300–2000, AR(1) 0/0.3/0.6). Our cascade is scikit-learn histogram boosting, not the released xgboost.

SPOT-cal floor: 6×10^{-3}

Largest in a fixed grid that pins at most 1% of true edges (held-out simulated graphs).

Released floor: 10^{-3}

From the released xgboost code.

Free pairs on PetShop

118 (SPOT-cal), 203 (SPOT), all (ABIC)

Learning method: how ABIC fits the graph

Parameter and structure learning in one optimization (shared by ABIC and SPOT)

- **Objective:** least-squares fit of the data to the ADMG plus a smooth ℓ_0 penalty that discourages extra edges
- **Constraints:** the graph must be acyclic and ancestral, enforced by an augmented Lagrangian
- **Solver:** L-BFGS-B on the directed weights W; error covariance Ω updated with RICF
- **Output:** W, Ω, hence directed and bidirected edges

Why this learner

Differentiable ADMG learning handles latent confounders without testing every conditional independence, but costs time as d^2 grows. That cost is what SPOT targets.

Compared fairly

Same optimizer, objective, restarts, thresholds and seeds for ABIC and SPOT; only the posterior differs.

Inference method: predicting one service from others

Exact Gaussian conditioning on the fitted graph

The fitted ADMG implies a joint Gaussian with covariance $\Sigma = (I - W)^{-T}\Omega(I - W)^{-1}$. For target s and evidence e:

$$\mu_{s|e} = \Sigma_{se}\Sigma_{ee}^{-1}x_e, \quad \sigma^2_{s|e} = \Sigma_{ss} - \Sigma_{se}\Sigma_{ee}^{-1}\Sigma_{es}$$

Exact for the fitted model; one linear solve per query, so fitting the graph, not the query, dominates runtime.

Two queries (E8)

A: predict the sink from its true ancestors.
B: predict the sink from all other services.

Checked against

True graph, DAG fits, a saturated Gaussian (no structure assumed) and a marginal (no structure used). Scored by held-out NLL (how surprised the model is by the true value; lower is better) and 90% interval coverage; both are defined in the metrics slides.

Outline

1. Problem and Model

2. Method

3. Setup and Hypotheses

4. Results

5. Conclusions

Setup: simulated ADMGs and four PetShop scenarios

Known truth on simulations; inverted call graph on PetShop

Simulated (known truth)

- $d = 25$; $n = 200, 500, 1000, 2000$
- E1: 10 random graphs per n
- E2: robustness at $n = 1000$ (density, noise, AR(1), 5% and 20% missing)
- Linear-Gaussian data

PetShop (Hardt et al.)

- 4 scenarios: low, high, temporal 1, temporal 2
- 29–35 observed and 9–13 latent services; $n = 589$ or 1,652
- Truth: inverted call graph, latent projection, PAG
- One series per scenario; 5 optimizer seeds

Compared

ABIC · SPOT (10^{-3}) · SPOT-cal (6×10^{-3}) · FCI and RFCI (Tetrad)

Five hypotheses were fixed before the experiments

Decision rule: all three criteria must hold

- **H1** SPOT improves skeleton F1 over ABIC on simulated ADMGs, and the gain shrinks with n
- **H2** The PetShop gain is smaller than the synthetic gain
- **H3** The speed-up comes mainly from the hard floor
- **H4** The posterior is calibrated on simulated data and over-confident on PetShop
- **H5** A fitted ADMG predicts held-out data better than no structure; accurate structure helps root-cause analysis

Decision rule

A difference is “supported” only if

1. paired t-test $p < 0.05$
2. Wilcoxon $p < 0.05$
3. 95% CI excludes 0

Rule fixed before the results; every comparison is paired.

Metrics (1/3): did we find the right graph, and at what cost?

Structure and cost metrics, defined before they appear in results

Metric	What it measures	Better
Precision, recall	Precision: share of edges we found that are real. Recall: share of real edges we found	higher
F1	Combines the two (their harmonic mean): high only if both are high	higher
Skeleton F1	F1 on *whether* a pair is connected, ignoring arrow directions	higher
Arrowhead, tail F1	F1 on edge directions (arrowhead or tail marks on the PAG)	higher
SHD (structural Hamming distance)	Number of edge differences (missing, extra, wrongly marked) between our PAG and the true PAG; read beside F1, because an empty graph can have a low SHD	lower
Convergence, time	Whether the optimizer finished inside a fixed time budget, and the seconds it used	converged, lower
Free pairs	Candidate edges left after pruning: how big the search is	fewer

Metrics (2/4): are the posterior's probabilities trustworthy?

How good is the learned edge probability? (headline: log-loss)

Metric	What it measures	Better
Log-loss	For edge / no-edge predictions: $-\log$ of the probability given to the correct answer, averaged. Punishes confident mistakes hard. Our headline metric	lower
Brier score	Mean squared gap between predicted probability and the true 0/1 outcome	lower
ECE (calibration error)	Group pairs by predicted probability; average gap between predicted and observed edge frequency	lower, near 0
AUROC, AUPRC	Does the posterior rank real edges above non-edges? (0.5 = chance, 1 = perfect). AUPRC suits rare edges	higher

Metrics (3/4): is the fitted model's prediction trustworthy?

Held-out prediction of a service's latency, and root-cause ranking

Metric	What it measures	Better
NLL (negative log-likelihood)	For predicting a service's latency: $-\log$ of the probability density the model gave to the value that actually occurred, averaged over held-out data, in nats. Low NLL means the model was rarely surprised	lower
Coverage	Share of true values falling inside the model's 90% prediction interval	near 0.90
Variance ratio	Actual squared error divided by the predicted variance; 1 means the stated uncertainty is right, above 1 means too confident	near 1
MRR	Root-cause ranking: average of 1/rank of the true culprit (rank 1 gives 1.0)	higher

Metrics (4/4): how we decide a difference is real

Statistical terms used on every results slide

Term	Meaning
Mean $\pm$ SD	Average over runs, and how much runs spread around it (standard deviation)
Paired comparison	For each graph or scenario and seed, subtract ABIC's score from SPOT's; we then study these differences, so easy and hard cases cancel
95% CI (confidence interval)	A range of values for the true average difference that the data cannot rule out. If it excludes 0, the difference is unlikely to be chance alone
p-value	Probability of a difference at least this large if the two methods were truly equal; small (< 0.05) means surprising under "no difference"
Paired t-test	Tests whether the average paired difference is zero; assumes the differences are roughly bell-shaped
Wilcoxon signed-rank test	Same question using only the ranks of the differences; no bell-shape assumption, but with 5 pairs its smallest possible p is about 0.06
Bootstrap (block)	Resample the data many times (PetShop in blocks of consecutive time points) and recompute the result, to see how much it would vary with other data
"Supported" (our rule)	t-test $p < 0.05$ and Wilcoxon $p < 0.05$ and 95% CI excludes 0, all three, fixed before the results

Statistics: what this design can and cannot show

Known limits, stated before the results

- 331 comparisons in total; single p-values are not headlines
- PetShop has one series per scenario: seeds measure the optimizer, not new data
- The 40 synthetic units reuse graphs; per-n tests are cleaner
- Scalability: 3–4 graphs per size, descriptive only

Consequence

No single PetShop scenario can pass the rule alone with five seeds. We pool 20 runs and say so.

Outline

1. Problem and Model

2. Method

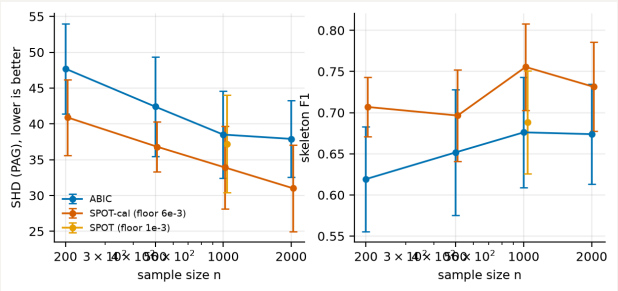
3. Setup and Hypotheses

4. Results

5. Conclusions

Simulated graphs: SPOT-cal gains +0.067 skeleton F1

E1: $d = 25$, 10 graphs per n (higher is better)



PAG SHD at $n = 1000$: 38.5 $\rightarrow$ 33.9. The released floor 10^{-3} gains only +0.012 $[-0.024, +0.048]$, not supported.

+0.067

pooled skeleton F1 gain, 95% CI [+0.044, +0.090];

31 wins, 9 losses

3 of 4

sample sizes supported

(200, 1000, 2000), not $n = 500$

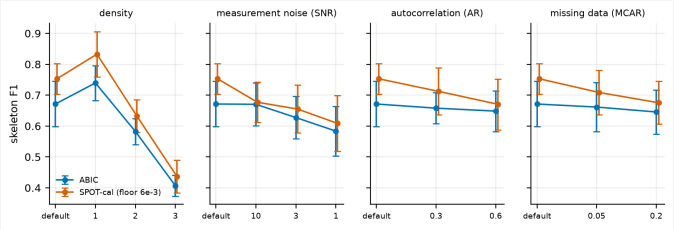
$p = 0.84$

no evidence that the gain shrinks with n

(interaction test)

SPOT-cal leads in all 10 robustness conditions

E2: robustness at $n = 1000$; gain supported in 6 of 10 conditions



Supported

density 1, 2, 3; SNR 1, 3; AR 0.3

Not supported

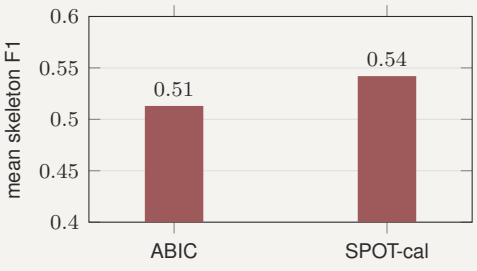
SNR 10, AR 0.6, 5% and 20% missing data

Sparsest graphs gain most

+0.093 at density 1, then +0.049 and +0.030

PetShop accuracy gain is small: +0.029 skeleton F1

E3: five optimizer seeds per scenario



Mean of the four scenarios

+0.029

pooled over 20 runs, CI [+0.014, +0.045];

PAG SHD -3.9 $[-5.4, -2.4]$

0 of 4

scenarios pass the rule alone:

5 seeds cannot reach Wilcoxon $p < 0.05$

+0.022

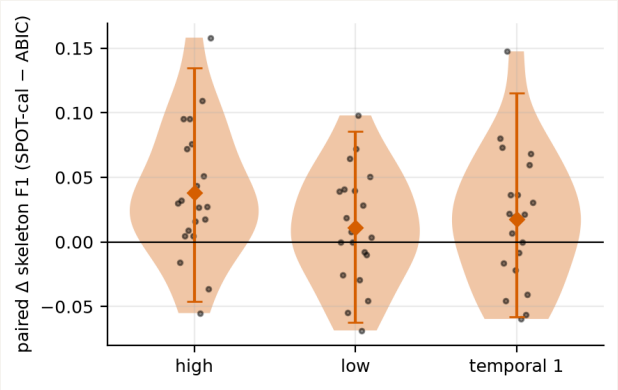
under block bootstrap (E5);

interval spans 0, not significant

PetShop gain minus simulated gain: -0.057 $[-0.097, -0.017]$ (H2 supported). Tail F1 is near zero for every method, so we make no orientation claim.

Resampling the data keeps the gain small and uncertain

E5: moving-block bootstrap of the PetShop time series, 3 scenarios



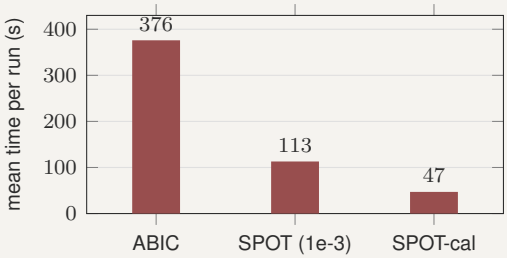
40 of 58

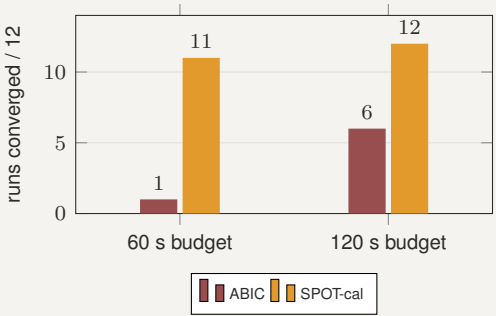
non-tied resamples favour SPOT-cal, mean +0.022, interval spans 0

The bootstrap resamples overlap, so we report the spread, not a p-value. We do not call the PetShop gain significant under resampling.

SPOT-cal is about 8× faster on PetShop

E6: optimizer time on PetShop

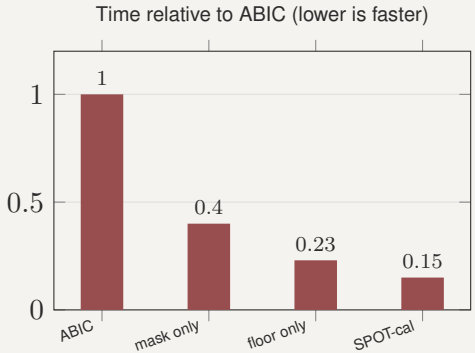


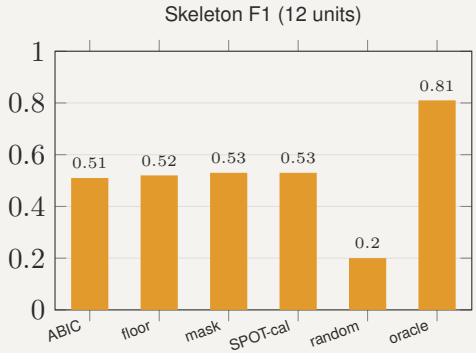


SPOT-cal was faster in 19 of 20 runs; paired difference -329 s, CI $[-454, -204]$. Left: main pass, run with 11 jobs at once, so compare ratios only. Right: quiet machine, 12 runs per cell.

The floor gives the speed; the posterior caps accuracy

Ablation by posterior variant

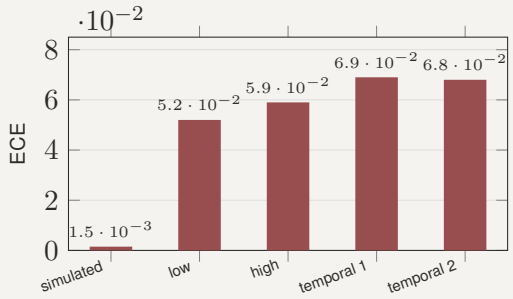




Floor alone recovers a median 95% of the saving (H3 partly supported). Only random (-0.316) and oracle ($+0.302$) differ from ABIC on F1.

Posterior calibration breaks on PetShop

H4: calibrated on simulations, over-confident on PetShop (ECE, lower is better)



Isotonic and Platt recalibration do not reduce PetShop ECE, and isotonic worsens log-loss.

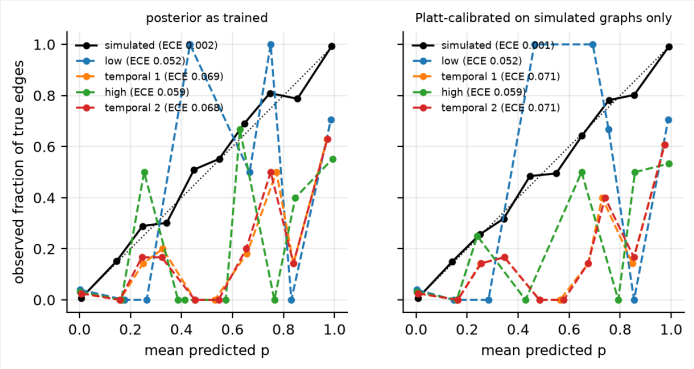
0.990 → 0.84–0.92
AUROC, simulated graphs to the four PetShop scenarios

+0.05
over-confidence on every PetShop scenario, all CIs above 0

0.54 → 0.81
skeleton F1 on PetShop if the posterior were an oracle

Reliability: predicted vs observed edge frequency

E7: the posterior's probabilities, simulated graphs vs PetShop

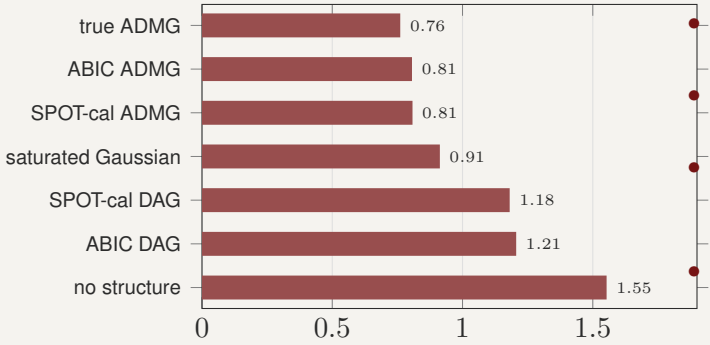


- Log-loss 0.052 simulated; 0.26–0.32 PetShop
- Brier 0.0136 simulated; 0.053–0.067 PetShop
- Points below the diagonal mean over-confident

PetShop intervals treat pairs as independent, so they are too narrow.

A fitted ADMG predicts held-out latency best

E8: held-out NLL, query B (lower is better); ADMG beats DAG and no structure



- +0.75 nats over no structure for both learned ADMGs, supported
- Beats DAGs by +0.40 (ABIC) and +0.37 (SPOT-cal), supported
- Saturated Gaussian: non-inferior only for the true graph (-0.031 vs margin -0.05)
- Coverage 0.925; variance ratio 1.43–1.46 (variance slightly under-estimated)

Query B predicts the sink service from all other variables. Rolling time-ordered splits, 20 units per model.

Classical FCI/RFCI: more accurate, about 40× faster

E9: skeleton F1 on simulated graphs, $n = 1000$ (α in labels)



+0.08 to +0.18
FCI/RFCI over SPOT-cal on all 10 graphs at every n and α

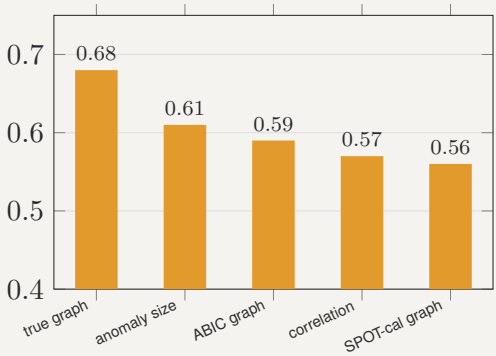
≈ 3.3 s
FCI/RFCI per graph, against 127 s (SPOT-cal) and 181 s (ABIC)

≈ 0.5 on PetShop
all methods; FCI 0.50, RFCI 0.51–0.52, ABIC 0.51, SPOT-cal 0.54

Linear-Gaussian data is the setting the Fisher-z test is designed for. We did not isolate why FCI and RFCI win.

Learned graphs did not improve root-cause ranking

E12: mean reciprocal rank (higher is better); the true graph did help



- True graph: +0.068 [0.001, 0.149] over anomaly size
- Learned graphs: ABIC -0.017, SPOT-cal -0.044, not significant
- True beats learned: +0.084 over ABIC, +0.112 over SPOT-cal
- Test split: SPOT-cal ranks worse than correlation, -0.148 [-0.297, -0.009]

Structure helps only when it is accurate

36 injected-issue cases from PetShop; 26 in the test split.

Scalability: lead kept from $d = 25$ to 100

Skeleton F1, $n = 1000$, 600 s budget; $d = 200$ did not complete (descriptive only)

	$d = 25$	$d = 50$	$d = 100$
ABIC (scaled start)	0.677	0.692	0.708
SPOT-cal	**0.763**	**0.759**	**0.753**

No p-values. $d = 200$: the workers hung and were stopped, so we make no claim.

- Plain ABIC diverges at $d = 100$ on 3 of 3 graphs (empty graph, F1 0)
- F1 gain on every seed: +0.087, +0.067, +0.044; SHD lower by 6, 12, 49
- At $d = 100$ both working variants hit the 600 s budget
- Posterior step: 0.4 s at $d = 25$, 5.2 s at $d = 100$

Encouraging but partial: $d = 200$ untested

Outline

1. Problem and Model

2. Method

3. Setup and Hypotheses

4. Results

5. Conclusions

Positioning against the SPOT paper and classical methods

Anchor: SPOT (Ma et al., KDD 2024); revisited after the experiments

What the anchor claims. Scalable differentiable causal discovery with latent confounders, by guiding an ABIC-style optimizer with a learned skeleton posterior.

What we did. Re-implemented optimizer and posterior, ran an independent paired evaluation on simulated graphs and PetShop, and added FCI/RFCI as classical baselines.

Repositioned claim

SPOT's speed-up is robust; its accuracy gain is real but small; it does not beat classical FCI/RFCI on our simulated graphs. We make no state-of-the-art claim.

Repositioning

From "SPOT is better" to "SPOT is faster, modestly more accurate, and not state of the art".

Scorecard: 2 supported, 3 partly supported

Verdict per pre-registered hypothesis

Hypothesis	Verdict	Key evidence
H1 Synthetic gain	**Partly supported**	Gain +0.067 shown; no shrinkage with n ($p = 0.84$)
H2 Smaller on PetShop	**Supported**	Gain difference -0.057 [-0.097, -0.017]
H3 Speed from the floor	**Partly supported**	Time ratio: floor 0.23, mask 0.40, both 0.15
H4 Posterior calibration	**Supported**	ECE 0.0015 simulated vs 0.052–0.069 PetShop
H5 Prediction and RCA	**Partly supported**	+0.75 nats over no structure; RCA gain only with true graph

What the evidence supports: a narrower claim

Narrower than state of the art: five questions, five honest answers

Question	Evidence	Answer
Beats its ABIC backbone?	Simulated +0.067 (supported); PetShop +0.029 (small)	✓ **Yes, modestly**
Faster?	About 8× lower time; 11/12 vs 1/12 converged at 60 s	✓ **Yes, the robust result**
Scales?	Ahead of scaled-start ABIC at $d = 25$–100; $d = 200$ not done	– **Encouraging, partial**
Better than classical?	FCI/RFCI F1 0.87–0.89 vs 0.756 on simulated graphs	✗ **No**
Helps downstream?	Better held-out prediction; no RCA gain from learned graphs	– **Prediction yes, RCA not shown**

What we learned

Lessons from building and testing the whole pipeline

1. **A prior mostly buys speed.** Fixing unlikely pairs to zero makes optimization about $8\times$ cheaper; accuracy gains are small.
2. **The posterior is the ceiling.** With an oracle posterior PetShop skeleton F1 would be 0.81, against 0.54 with ours.
3. **Calibration does not transfer for free.** Trained on simulated graphs, the posterior is over-confident on PetShop.
4. **Details decide.** The floor must be calibrated for the model: with the released floor the synthetic gain disappears. ABIC's default start diverges at $d = 100$.
5. **Structure helps only if accurate.** Learned graphs predict better than no structure but did not improve root-cause ranking; the true graph did.
6. **What did not work.** E2, E5 and $d = 200$ are incomplete; we do not make claims that are undecidable without them.

Limitations: to be understood before quoting the results

Data and design, work not completed, measurement

Data and design

- One dataset per PetShop scenario
- Seeds measure optimizer variance only
- Linear-Gaussian model, i.i.d. rows
- Floor tuned on simulated graphs

Not completed

- E2: 79 of 80 jobs
- E5: 3 of 4 scenarios
- E11 error analysis skipped
- Scalability stops at $d = 100$
- Exact vs approximate inference not run

Measurement

- Main-pass times taken under load; quote ratios
- E6 and E8 ran on a quiet machine
- Both methods add too many $\leftrightarrow$ edges: no orientation claim

Conclusion: faster, modestly more accurate

Limited by the posterior: headline results and next steps

$\approx 8\times$ faster
mean optimizer time on PetShop;

11/12 vs 1/12 converged at 60 s

+0.067 / +0.029
skeleton F1 gain, simulated graphs /

PetShop (pooled)

0.54 $\rightarrow$ 0.81
PetShop skeleton F1 with an oracle

posterior: the posterior is the ceiling

Next steps

1. Retrain the posterior on graphs up to $d = 200$ and retest
2. Close the oracle gap with PetShop-like training graphs
3. Repeat on more than one dataset per scenario
4. Try a classical skeleton prior (for example FCI)
5. Measure exact vs approximate inference cost

Thank you. Questions?

Reproducibility and references

Reproducibility

- Python 3.13, scikit-learn 1.9.1, Java 17+ with causal-cmd 1.3.0
- `bash run_all.sh main <workers>`, then `quiet`
- Seeds stored in every result row
- Every table and figure is generated from result files

Key references

- Ma et al., SPOT, KDD 2024
- Bhattacharya et al., ABIC, AISTATS 2021
- Hardt et al., PetShop, CLeaR 2024
- Zheng et al., NOTEARS, NeurIPS 2018
- Spirtes et al. 2000; Colombo et al. 2012 (FCI, RFCI)

Anant Narayan · **Ankit Kumar** · **Milan Shah**

Anant: SPOT implementation and report · Ankit: benchmarks · Milan: inference and video

Generative-AI assistance was used for code and report preparation; all team members verified the experiments, metrics, tests and conclusions.